

Grain Segmentation in exaStamp

A Workflow Reference

exaStamp / `src/grains`

August 4, 2026

Abstract

This document describes how exaStamp’s grain-segmentation pipeline works: from per-atom crystal orientation (as fitted by Polyhedral Template Matching, PTM [1]) to a per-atom grain id, a per-grain mean orientation, and a color suitable for visualization. The implementation follows OVITO’s own `GrainSegmentation` modifier (`crystalanalysis/modifier/grains`) directly: Node-Pair-Sampling graph clustering with an automatically selected merge threshold, and Dijkstra-by-distance orphan-atom adoption — chosen specifically over the simpler minimum-spanning-tree/majority-vote alternatives OVITO also offers, for robustness against thermally noisy samples. The pipeline lives in `src/grains/` and is exercised end to end by `data/regression_new/grains/compute_grain_segmentation.msp`, verified directly against real OVITO output on that same test case.

Contents

1	Overview	2
2	Input: PTM structure type and orientation	2
3	Bond disorientation	3
3.1	Disorientation angle	3
3.2	GPU-compatible bond recording	3
4	Node-Pair-Sampling clustering	4
4.1	The graph	4
4.2	Reciprocal-nearest-neighbor chain clustering	4
4.3	Orientation accumulation	5
5	Automatic threshold selection	5
6	Cutting the dendrogram	6
7	Orphan atom adoption	6
8	Output	6
9	Verification against real OVITO output	7
10	Known limitations	7

1 Overview

Given a per-atom crystal structure type and lattice orientation (from PTM), grain segmentation partitions the crystalline atoms into contiguous *grains* — regions of consistent orientation — separated by grain boundaries, and optionally extends each grain to absorb nearby non-crystalline (“Other”) atoms that sit within its own boundary region. The output, per atom, is a grain id (or none); per grain, an atom count, a mean lattice orientation, and a display color.

The pipeline has two stages, corresponding to two separate operators:

Operator	Role
<code>compute_grain_bond_misorientation</code>	GPU-compatible. For every chunk-neighbor bond, records the neighbor’s identity, real distance, and (for a same-structure-type pair) the crystal-symmetry-aware disorientation angle.
<code>compute_grain_clusters</code>	Host, sequential. Node-Pair-Sampling clustering with auto-threshold selection, dendrogram cut, minimum-grain-size dissolution, optional Dijkstra-by-distance orphan adoption, and per-grain color assignment. Writes the final <code>grain_id</code> / <code>grain_color</code> fields.

Table 1: The two grain-segmentation operators.

This split mirrors OVITO’s own two-phase engine design (`GrainSegmentationEngine1`: neighbor bonds and the dendrogram, expensive; `GrainSegmentationEngine2`: threshold cut and coloring, cheap) for the same reason: bond disorientation is embarrassingly parallel and belongs on the GPU, while dendrogram construction and orphan adoption are inherently sequential (each step depends on the previous one) and are not GPU-parallelizable in OVITO’s own implementation either.

2 Input: PTM structure type and orientation

Both operators consume two per-particle fields already produced upstream by `compute_ptm + ptm_fields`:

- `ptm_type`: a discrete structure-type code (none, FCC, HCP, BCC, ICO, SC, ...),
- `ptm_orientation`: the fitted lattice orientation, stored as a rotation matrix.

PTM itself fits this orientation via Theobald’s QCP (quaternion-characteristic-polynomial) method together with combinatorial template/graph matching for the atom-to-template correspondence [1] — a well-validated, established registration method, not something this pipeline re-derives.

Only two symmetry families are supported for clustering, matching OVITO’s own `GrainSegmentation` scope exactly: **cubic** (FCC, BCC, SC — the 24-element proper rotation point group) and **HCP** (the 12-element “conventional” hexagonal group). Icosahedral structure has no supported symmetry group in OVITO’s own grains code either, and is therefore never merged into a grain by this pipeline.

Ghost-atom copies of these two fields must be explicitly synchronized (`ghost_update_opt`) before either grain-segmentation operator runs, since bonds routinely cross into a neighboring rank’s ghost territory.

3 Bond disorientation

3.1 Disorientation angle

For two atoms of the *same* structure type with fitted orientation quaternions $\hat{q}_0$ and $\hat{q}_1$ (converted on the fly from the stored rotation matrix, via a numerically robust, branch-selected matrix-to-quaternion formula), the *disorientation* is the smallest possible misorientation angle between the two orientations once every crystallographically equivalent representation of each is accounted for:

$$\theta = \min_{g \in G} \angle(\hat{q}_0^{-1} \hat{q}_1 g), \quad (1)$$

where G is the crystal’s own point-group symmetry group (24 elements for cubic, 12 for HCP) acting on the relative orientation quaternion $q_{\text{rel}} = \hat{q}_0^{-1} \hat{q}_1$. Concretely: rotate q_{rel} into its own *fundamental zone* by finding the symmetry element g^* maximizing $|q_{\text{rel}} \cdot g|$ (the crystallographically equivalent orientation closest to the identity), then

$$\theta = \arccos(2t^2 - 1), \quad t = (q_{\text{rel}} g^*)_0, \quad (2)$$

the scalar (real) component of the fundamental-zone-rotated relative quaternion — the standard double-angle identity $\cos \theta = 2 \cos^2(\theta/2) - 1$ applied to a unit quaternion’s own scalar part, with the squaring removing the quaternion double-cover sign ambiguity. A bond between two atoms of *different* structure types, or where either endpoint has no structure classification at all, has no defined disorientation.

This formula, together with the two symmetry-group tables it uses, is ported verbatim from the same third-party PTM library [1] OVITO’s own `GrainSegmentation` code calls (`ptm_quat.cpp`) — already vendored elsewhere in `exaStamp` for PTM itself, but duplicated here as small, explicitly device-annotated inline functions rather than linked, specifically so this pass can be genuinely GPU-compatible (the vendored library’s own functions are plain host C++, not written for device compilation).

3.2 GPU-compatible bond recording

`compute_grain_bond_misorientation` visits every chunk-neighbor bond of every owned atom and, for each one, records the neighbor’s global id, the real Euclidean bond length, and — if both atoms share a supported structure type and their disorientation clears a hard ceiling (`misorientation_threshold`, 4° by default, matching OVITO’s own hardcoded value) — the disorientation angle itself; otherwise a sentinel value marks the bond as ineligible for clustering. Every atom’s own set of bonds is capped at a small, fixed number of slots (24); when a new bond arrives after the cap is reached, it *replaces* whichever currently-recorded bond is farthest away, so the retained set is always the nearest bonds seen so far, not merely the first ones encountered in traversal order. This matters for two separate downstream consumers: a dropped near bond can be the only real candidate for clustering, and — more consequentially — it can be the only short path an orphan atom needs to reach a grain (§7).

The recorded bond list therefore serves two distinct downstream purposes at once, exactly as in OVITO’s own engine (which reuses one `_neighborBonds` array for both): the disorientation value gates which bonds are *crystalline clustering candidates* (§4), while the distance value, together with every recorded bond regardless of disorientation validity, feeds orphan-atom adoption (§7).

4 Node-Pair-Sampling clustering

4.1 The graph

Every candidate crystalline bond (disorientation below threshold) becomes one edge of an undirected weighted graph over atoms, with weight

$$w = \exp(-\theta^2/3), \quad (3)$$

a Gaussian decay in the disorientation angle θ (in degrees): a perfectly aligned bond gets weight 1, and weight falls off smoothly as misorientation grows toward the 4° ceiling.

4.2 Reciprocal-nearest-neighbor chain clustering

Node-Pair-Sampling is a form of hierarchical agglomerative clustering, implemented as a reciprocal-nearest-neighbor (RNN) chain — the classical technique that avoids re-scanning the whole graph after every single merge. For a node a , its own nearest neighbor is

$$\text{NN}(a) = \underset{v \in \text{adj}(a)}{\text{argmin}} \frac{\text{wnode}(v)}{w(a, v)}, \quad (4)$$

where $\text{wnode}(v)$ is the sum of edge weights incident on v — a node with many strong edges elsewhere (already part of a well-formed grain) is a *less* attractive nearest neighbor than one with few, concentrating merges where the local evidence is strongest. Algorithm 1 sketches the chain-following procedure.

Algorithm 1 Reciprocal-nearest-neighbor chain clustering

```

1: while active nodes remain do
2:    $chain \leftarrow$  [any active node]
3:   while  $chain$  is not empty do
4:      $a \leftarrow chain.pop()$ 
5:      $(d, b) \leftarrow \text{NN}(a)$ 
6:     if  $b$  does not exist then                                 $\triangleright a$ 's own connected component is fully clustered
7:       deactivate  $a$ 
8:     else if  $chain$  is not empty then
9:        $c \leftarrow chain.pop()$ 
10:      if  $c = b$  then                                           $\triangleright a$  and  $b$  are mutual nearest neighbors
11:        merge  $a$  and  $b$  (contract the smaller-degree node into the larger)
12:        accumulate the absorbed node's own orientation into the survivor (§4.3)
13:        record a dendrogram entry at distance  $d$ 
14:         $chain.push(\text{merged node})$ 
15:      else
16:         $chain.push(c)$ ;  $chain.push(a)$ ;  $chain.push(b)$ 
17:      end if
18:    else
19:       $chain.push(a)$ ;  $chain.push(b)$ 
20:    end if
21:  end while
22: end while

```

Following the chain — rather than always restarting from scratch — guarantees that when two nodes finally *are* found to be mutual nearest neighbors, merging them immediately is always a valid, myopically-optimal step; a node whose own nearest neighbor keeps changing as the chain extends simply gets pushed further down the chain until a genuine mutual pair is found

somewhere along it. This scales far better than repeatedly rescanning every node’s own nearest neighbor from scratch after each individual merge.

Implementation note. exaStamp’s own adjacency structure is a plain hash map per node (neighbor $\rightarrow$ edge weight), in place of OVITO’s own intrusive red-black tree plus preallocated edge buffer — the same algorithm and the same clustering result, at the cost of a somewhat less memory-efficient constant factor that is a complete non-issue at ordinary atomic coordination numbers (~ 12 – 18 neighbors per atom).

Why Node-Pair-Sampling, not a minimum spanning tree. OVITO also offers a simpler minimum-spanning-tree (Kruskal, single-linkage) clustering mode. Single-linkage clustering is prone to *chaining*: a handful of thermally noisy, low-misorientation bonds can bridge two genuinely distinct grains into one cluster via a cheap path, even when the bulk of their own atoms disagree strongly. The RNN-chain contraction above behaves more like average-linkage clustering (edge weights accumulate as clusters merge, so a merge decision reflects the accumulated evidence of an entire growing cluster, not one lone bond) — considerably more resistant to this failure mode, which is precisely why it was chosen here for studying samples with real thermal noise.

4.3 Orientation accumulation

Each atom begins as its own single unit quaternion. At every merge, the absorbed cluster’s own running orientation sum is first *mapped* onto the surviving cluster’s current orientation neighborhood — the same fundamental-zone search used for the disorientation formula itself, applied here to find which crystallographically equivalent representation of the absorbed orientation is actually closest to the survivor — and then accumulated, weighted by its own pre-mapping norm:

$$\mathbf{q}_{\text{parent}} += \text{map}(\mathbf{q}_{\text{parent}}, \mathbf{q}_{\text{child}}) \cdot \|\mathbf{q}_{\text{child}}\|. \quad (5)$$

Since every atom starts as a unit quaternion and quaternions are never renormalized between merges, the norm of a cluster’s own running sum equals exactly the number of atoms merged into it so far — a compact way to implement a symmetry-aware running weighted quaternion mean without ever re-visiting member atoms. The final per-grain orientation reported downstream is this running sum, normalized once at the very end.

5 Automatic threshold selection

The RNN-chain clustering produces a full dendrogram (every merge, in the order it happened, together with the distance d at which it occurred and the size of each side being merged); the question of *where* to cut it — which merges represent genuine intra-grain noise and which represent crossing a real grain boundary — is answered by a robust statistical fit, not a user-chosen fixed threshold, following OVITO’s own default (`GraphClusteringAutomatic`) mode exactly.

Within a single, well-formed grain, successive merges’ own distances tend to grow smoothly and predictably with the size of the clusters being merged (more atoms already agreeing makes further agreement less surprising); a merge that instead crosses a real grain boundary shows up as an *anomalous jump* away from that trend. Concretely: fit a power law $\log d \approx \gamma \log s + c$ (where s is the harmonic-mean merge size) via robust, iteratively reweighted least-absolute-deviations regression (100 fixed iterations, weight update $w \leftarrow s / \max(10^{-4}, |\text{residual}|)$ at each iteration); the threshold is then the *largest* $\log d$ among merges still consistent with that fitted trend (residual below $1.5\times$ the median absolute residual) — i.e. the last merge that still looks like ordinary intra-grain noise. Every merge above that threshold is presumed to be a genuine inter-grain boundary crossing and is excluded when the dendrogram is cut.

A manual threshold (OVITO’s `GraphClusteringManual` mode) is also exposed, expressed in this same internal log-distance unit rather than a physically interpretable angle — this is an

intentional fidelity choice, matching how OVITO’s own manual mode for this particular clustering algorithm works, not a simplification introduced here.

6 Cutting the dendrogram

Applying the threshold is a single linear pass over the (distance-sorted) dendrogram, replaying each accepted merge through a union-find structure until the first merge whose own $\log d$ exceeds the threshold. Every resulting connected component below `min_grain_atom_count` (100 atoms by default, matching OVITO’s own default) is dissolved back to “no grain” rather than kept as a spuriously tiny cluster. Surviving components are assigned contiguous, 1-based grain ids, ordered largest-to-smallest by atom count, and each is assigned a random display color drawn from a fixed-seed HSV distribution (seed 1, matching OVITO’s own hardcoded seed exactly, so repeated runs give reproducible, comparable colors).

7 Orphan atom adoption

An atom with no PTM structure classification at all (sitting inside a defect core or a genuinely disordered region) — or a crystalline atom whose own cluster was dissolved for being too small — has no grain id after the steps above. Orphan adoption (optional, on by default) assigns each such atom to whichever grain is geometrically closest, using every recorded bond (§3), not only the crystalline-candidate subset.

This is a single-pass, multi-source Dijkstra: a min-priority queue is seeded with every bond connecting an already-assigned atom to an unassigned neighbor, weighted by real bond length; the queue is repeatedly popped, and whichever unassigned atom is reached by the cheapest accumulated path is settled to that path’s own originating grain and used to extend the search further, exactly as in an ordinary shortest-path computation with multiple sources. An atom is therefore adopted by the grain that is closest by cumulative real-space path length through the network of non-crystalline/boundary bonds — not by a majority vote among its own immediate neighbors, and not by straight-line distance.

A real bug found via a boundary-region visual inspection, and its fix. Because bond recording (§3) only ever runs on owned (not ghost) atoms, a ghost atom’s own “outgoing” bond list is always empty — even when that ghost carries a perfectly valid grain id (as the periodic image of an already-clustered atom). Seeding and propagating the priority queue directly from each atom’s own one-directional recorded bond list therefore could never adopt an orphan whose only nearby assigned neighbor happened to be a ghost, i.e. exactly the atoms sitting right at a periodic domain boundary. The fix builds one *symmetric* adjacency structure from every recorded bond (both directions, regardless of which atom happened to record it) before running Dijkstra, removing the directional gap entirely. On the staged 28k-atom, 8-grain test case, this raised orphan coverage from ~90% (2913 atoms permanently stuck at boundaries, median distance to the nearest domain face 0.44 Å versus 4.67 Å over all atoms) to a full 100%.

8 Output

`compute_grain_clusters` writes two per-atom fields: `grain_id` (integer-valued, -1 for “no grain”, matching this project’s own established convention for a similar field in the dislocation-extraction pipeline) and `grain_color` (RGB, looked up from the owning grain’s own assigned color, or a fixed neutral gray for unassigned atoms). Both are ordinary named grid fields and can be written by `write_xyz` or visualized directly in ParaView like any other per-particle field.

9 Verification against real OVITO output

The staged regression test (`data/regression_new/grains/`) is a synthetic 28,498-atom Ta polycrystal built from 8 randomly oriented grains in an 80 Å cubic box. Run with `min_grain_atom_count=100` and orphan adoption enabled, and compared directly against a real OVITO run on the identical input with the same parameters:

- **8/8 grains** — exact match to the true grain count.
- **100% agreement on assigned-vs-unassigned**: all 28,498 atoms are either grain-assigned in both outputs or in neither.
- Per-grain sizes agree with OVITO’s own to within **1–6%** (this implementation: 5539/4574/3818/4080/3988 atoms; OVITO: 5494/4561/4066/4021/3827/3596/1547/1386) — a close match given the intentionally simplified (not byte-identical) adjacency data structure used for clustering.

10 Known limitations

- **Coherent-interface handling.** OVITO’s own optional pre-clustering pass that reclassifies a coherent twin boundary or stacking fault (locally a different but closely related structure, e.g. HCP inside an FCC matrix) into the surrounding grain’s own phase before clustering is not implemented here; every such interface is currently treated as an ordinary grain boundary.
- **Cross-rank MPI grain stitching.** Each MPI rank currently clusters its own local (owned + ghost) view of the system independently. A grain whose own true extent spans an MPI rank boundary is *not* reconciled into one consistent id across ranks — the same scope gap already documented for this project’s dislocation-extraction pipeline’s own per-rank cluster-building stage. Not yet tested at more than one MPI rank at all.

References

- [1] P. M. Larsen, S. Schmidt, J. Schiøtz, *Robust structural identification via polyhedral template matching*, Modelling and Simulation in Materials Science and Engineering **24** (2016) 055007.